

Reviewer Pack — Minimal Rank of Tropicalized Tensor Products vs Resolution P...

Entry 852aae70e60d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 22:34:37 UTC

Minimal Rank of Tropicalized Tensor Products vs Resolution Proof Width

Entry ID: 852aae70e60d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 22:34:37 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Tensor Products) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{'text': 'For every instance of a Tseitin formula F with n variables and m clauses, the minimal rank of its tropicalized tensor product is upper-bounded by the width of the resolution proof tree for F . Specifically, if F has a resolution proof tree with width $w(F)$, then the minimal rank of the tropicalization of the tensor product of the clauses in F is $O(w(F))$.', 'counterexample': 'An instance F with a resolution proof tree of width less than $2 * \text{minimal rank}(\text{tropicalized tensor product of clauses in } F)$.'}.

Rationale (proposer's reasoning):

{'text': 'Tropical geometry provides a way to study the structure of algebraic objects over the tropical semiring, which may reveal hidden structures in complex computational problems. The use of tropical tensor products could expose non-trivial relationships between the complexity of resolution proofs and the geometric properties of the clauses.', 'constructive_mapping': 'The constructive mapping transforms each clause c_i of F into a vector v_i in $\mathbb{R}^{n+1}$ by evaluating c_i at all possible assignments to its variables, then compute the tensor product of these vectors.'}

Taxonomy category: TROPICAL_GEOMETRY_TENSOR_PRODUCTS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c564dd7664d6ef16

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the ratio of the minimal rank of the tropicalized tensor product to the width of the resolution proof tree for all Tseitin formulas F with n variables and m clauses is less than or equal to 1.0, calculated over a statistically significant sample size of at least 100 random formulas.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Tropical Geometry AND Resolution Proof Complexity - Tropicalized Tensor Products IN Tropical Geometry AND Resolution Proof Width IN Complexity Theory - Resolution Proof Tree Width vs Minimal Rank Tropicalization Tensor Products

Top relevant hits considered: - [http://arxiv.org/abs/1206.1925v1] Counting Algebraic Curves with Tropical Geometry - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/2309.17302v2] Geometry of tropical extensions of hyperfields - [http://arxiv.org/abs/1407.1593v2] A Constructive Algorithm for Decomposing a Tensor into a Finite Sum of Orthonormal Rank-1 Terms - [http://arxiv.org/abs/2509.09463v1] Minimality of Tree Tensor Network Ranks - [http://arxiv.org/abs/1204.1444v2] Computing the minimum rank of a loop directed tree

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n, m):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []

        # Generate literals
        literals = [f'{var}' for var in variables] + [f'~{var}' for var in variables]

        # Generate clauses
        for _ in range(m):
            clause = random.sample(literals, 2)
            if random.choice([True, False]):
                clause[0] = f'~{clause[0]}'
                clause[1] = f'~{clause[1]}'
            clauses.append(clause)
```

```

# Generate the Tseitin formula
tseitin_formula = []
for i, clause in enumerate(clauses):
    new_var = f'y{i+1}'
    tseitin_formula.append([f'{new_var}', *clause])
    tseitin_formula.append([f'~{new_var}', f'~{clause[0]}', f'~{clause[1]}'])

# Add the final clause
tseitin_formula.append(['z'] + [f'~{var}' for var in variables])

return variables, clauses, tseitin_formula

def tropicalize(tensor):
    n = len(tensor)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            result[i][j] = max(tensor[i][k] + tensor[k][j] for k in range(n))
    return result

def rank(matrix):
    m, n = len(matrix), len(matrix[0])
    if m > n:
        matrix = list(zip(*matrix))
        m, n = n, m

    for i in range(m):
        max_row = max(range(i, m), key=lambda r: abs(matrix[r][i]))
        if matrix[max_row][i] == 0:
            return float('inf')

    # Swap rows
    matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

    # Eliminate below
    for j in range(i+1, m):
        factor = -matrix[j][i] / matrix[i][i]
        for k in range(n):
            matrix[j][k] += factor * matrix[i][k]

    return sum(1 for row in matrix if any(row))

def resolution_width(clauses):
    queue = clauses.copy()
    width = 0

    while queue:
        new_clause = []
        for clause in queue:
            if len(clause) == 1:
                return float('inf')

            literal, *rest = clause
            if literal.startswith('~'):
                continue

            new_clause.append(literal)
            new_clause.extend(rest)
            queue.append(new_clause)

```

```

        for other_clause in queue:
            if literal in other_clause and any(l.startswith('~') for l in rest):
                new_literal = [l for l in rest if not l.startswith('~')]
                new_clause.append(new_literal)

        width = max(width, len(queue))
        queue = new_clause

    return width

n = random.randint(5, 40)
m = random.randint(n, n*2)
variables, clauses, tseitin_formula = generate_tseitin_formula(n, m)

tensor_product = [[0] * (n+1) for _ in range(n+1)]
for clause in tseitin_formula:
    for literal in clause:
        if literal.startswith('~'):
            var_index = variables.index(literal[1:]) + 1
            tensor_product[var_index][var_index] = float('-inf')
        else:
            var_index = variables.index(literal) + 1
            tensor_product[0][var_index] = 1

tropicalized_tensor = tropicalize(tensor_product)
minimal_rank = rank(tropicalized_tensor)

resolution_tree_width = resolution_width(clauses)

conjecture_holds = minimal_rank <= resolution_tree_width
counterexample = "" if conjecture_holds else f"An instance F with a resolution proof tree of w

return {
    "metric_name": "Minimal Rank vs Resolution Width",
    "metric_value": minimal_rank / resolution_tree_width,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) if len(sys.argv) > 1 else [2**i for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_730cdd59.py", line 138, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_730cdd59.py", line 104, in run_trial
    variables, clauses, tseitin_formula = generate_tseitin_formula(n, m)
                                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_730cdd59.py", line 26, in generate_tseitin_formula
    literals = [f'{var}'] + [f'~{var}' for var in variables]
               ^^^
NameError: name 'var' is not defined. Did you mean: 'vars'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from verifying the conjecture's support or falsification. | next: Debug the test code to ensure it runs successfully and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13221
2	propose	ollama_remote	glm4:latest	0	0	14439
3	propose	ollama_remote	glm4:latest	0	0	12349
4	preregistration	ollama_remote	glm4:latest	0	0	5404
5	novelty	ollama_remote	glm4:latest	0	0	4588
6	novelty	ollama_remote	glm4:latest	0	0	5946
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38085
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10522

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10565
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14721
11	judge	ollama_remote	glm4:latest	0	0	17423

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 147265 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/852aae70e60d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/852aae70e60d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/852aae70e60d.tar.gz (if generated)